

Java Interview - Most Asked Questions

What's Covered (At a Glance)

#	Topic	Questions	Frequency Level
1	Core Java Basics	12	Every interview
2	OOPs	10	Every interview
3	Collections Framework	10	Every interview
4	Multithreading & Concurrency	10	Very High
5	Exception Handling	6	High
6	Java 8+ Features	8	Very High
7	JVM & Memory	6	High
8	Design Patterns	6	Very High
9	SOLID Principles	5	High
10	Tricky Output Questions	5	Every interview
	Total	78	

Open in Laptop/PC for better view.
Feel Free To DM @Kaash.sde for any doubts or query.

Core Java Basics (Top 12)

#	Question	Category	Frequency
1	JDK vs JRE vs JVM - difference explain karo	Platform	Every interview
2	Java platform independent kyun hai?	Platform	Very High
3	== vs .equals() - kab kya use karein?	Comparison	Every interview
4	String immutable kyun hai Java me?	Strings	Very High
5	String vs StringBuilder vs StringBuffer - difference?	Strings	Very High
6	String Pool kya hai? "hello" == "hello" kyun true hai?	Memory	High

#	Question	Category	Frequency
7	final, finally, finalize - teeno me difference?	Keywords	Very High
8	static keyword - variable, method, block, nested class?	Keywords	High
9	Wrapper classes kya hain? Autoboxing/Unboxing?	Types	High
10	Integer cache (-128 to 127) - kyun hota hai?	Memory	High
11	Pass by value vs Pass by reference - Java me kya hota hai?	Fundamentals	High
12	Type casting - widening vs narrowing?	Fundamentals	Medium

Key Answers

== vs .equals():

```
String a = "hello";
String b = "hello";
String c = new String("hello");

a == b // true (same object in String pool)
a == c // false (different objects - new creates on heap)
a.equals(c) // true (compares actual content)
```

Rule: == compares references (memory address). .equals() compares values. Objects ke liye hamesha .equals() use karo.

String immutable kyun hai:

1. Security - SQL queries, file paths, network connections me strings use hote hain. Mutable ho toh koi bhi change kar de.
2. String Pool - immutable hai toh same value share ho sakti hai. Memory bachti hai.
3. Thread safety - immutable objects inherently thread-safe hain.
4. Hashing - HashMap me key ki tarah use hota hai. Mutable ho toh hashCode badal

jayega. **final vs finally vs finalize:**

Keyword	Kya karta hai
final	Variable = constant, Method = override nahi ho sakta, Class = extend nahi ho sakti
finally	try-catch ke baad hamesha execute hota hai (cleanup ke liye)

finalize	GC object destroy karne se pehle call karta hai (deprecated - use mat karo)
----------	---

OOPs (Top 10)

#	Question	Category	Frequency
1	OOPs ke 4 pillars explain karo with examples	Core	Every interview
2	Abstraction vs Encapsulation - difference?	Core	Very High
3	Abstract class vs Interface - kab kya use karein?	Design	Every interview
4	Method overloading vs Method overriding - difference?	Polymorphism	Very High
5	Can we override static methods? Private methods?	Polymorphism	High
6	Multiple inheritance Java me kyun nahi hai?	Inheritance	High
7	Diamond problem kya hai?	Inheritance	High
8	super keyword - constructor chaining kaise kaam karta hai?	Inheritance	High
9	Object class ke methods batao (toString, equals, hashCode, clone)	Core	High
10	Composition vs Inheritance - kab kya prefer karein?	Design	High

Key Answers

Abstract class vs Interface:

Feature	Abstract Class	Interface
Methods	Abstract + concrete dono	Java 8+ me default methods aa gaye
Variables	Any type (instance, static)	Sirf public static final (constants)
Constructor	Haan	Nahi
Multiple	Ek hi extend kar sakte	Multiple implement kar sakte
Access	Any (private, protected, public)	Sirf public
Kab use karein	"IS-A" relationship + shared code	Contract define karna hai (WHAT, not HOW)

Composition vs Inheritance:

```
// Inheritance - "Car IS A Vehicle"
class Car extends Vehicle { }

// Composition - "Car HAS AN Engine"
class Car {
    private Engine engine; // Car contains Engine
}
```

Rule: "Favor composition over inheritance" - kyunki composition loose coupling deta hai. Inheritance se tight coupling hoti hai.

Collections Framework (Top 10)

#	Question	Category	Frequency
1	ArrayList vs LinkedList - difference aur kab kya use karein?	List	Very High
2	HashMap internal working explain karo	Map	Every interview
3	HashMap vs Hashtable vs ConcurrentHashMap - difference?	Map	Very High
4	HashSet internally kaise kaam karta hai?	Set	High
5	TreeMap vs HashMap vs LinkedHashMap - kab kya?	Map	High
6	Comparable vs Comparator - difference?	Sorting	Very High
7	fail-fast vs fail-safe iterators?	Iteration	High
8	How does HashMap handle collisions?	Map	Every interview
9	equals() aur hashCode() ka contract kya hai?	Hashing	Very High
10	PriorityQueue kaise kaam karta hai?	Queue	Medium

Key Answers

HashMap Internal Working:

1. HashMap internally array of Node (bucket) use karta hai
2. Key ka hashCode() calculate hota hai → index decide hota hai (hash & (n-1))

3. Us index pe Node store hota hai (key, value, hash, next)
4. Collision hone pe:
 - a. Java 7: Linked List ($O(n)$ worst case)
 - b. Java 8: 8+ nodes pe Red-Black Tree ban jaata hai ($O(\log n)$)
5. Load factor 0.75 - 75% full hone pe rehash (capacity double)
6. Initial capacity: 16

```
// Collision handling
```

```
Bucket[0] → Node(K1,V1) → Node(K2,V2) → Node(K3,V3) // Linked List
Bucket[0] → TreeNode(balanced) // After 8 nodes
```

equals() and hashCode() contract:

1. Agar `a.equals(b) == true` → `a.hashCode() == b.hashCode()` (MUST)
2. Agar hashCode same hai → equals true NAHI BHI ho sakta (collision)
3. Dono override karo agar HashMap/HashSet me key ki tarah use karna hai
4. Ek override kiya doosra nahi → bugs (HashMap entry dhundh nahi payega)

Multithreading & Concurrency (Top 10)

#	Question	Category	Frequency
1	Thread vs Runnable - kab kya use karein?	Basics	Very High
2	Thread lifecycle explain karo (states)	Basics	High
3	synchronized keyword kya karta hai?	Synchronization	Very High
4	Deadlock kya hai? Kaise avoid karein?	Problems	Every interview
5	volatile keyword kya karta hai?	Memory	High
6	wait() vs sleep() - difference?	Methods	Very High
7	ExecutorService kya hai? Thread pool kyun use karte hain?	Advanced	High
8	Callable vs Runnable - difference?	Advanced	High
9	CompletableFuture kaise kaam karta hai?	Async	High
10	ConcurrentHashMap internally kaise kaam karta hai?	Collections	High

Key Answers

Deadlock example:

```
// Thread 1: locks A, then tries to lock B
// Thread 2: locks B, then tries to lock A
// Both waiting forever - DEADLOCK

// Prevention:
// 1. Always lock in same order
// 2. Use tryLock with timeout
// 3. Avoid nested locks
// 4. Use higher-level concurrency utilities
```

wait() vs sleep():

Feature	wait()	sleep()
Class	Object	Thread
Lock release	Haan - lock chhod deta hai	Nahi - lock hold karta hai
Wake up	notify()/notifyAll() se	Time khatam hone pe
Where	synchronized block me hi	Kahin bhi
Purpose	Inter-thread communication	Pause execution

volatile:

```
// Without volatile - thread cached value use kar sakta hai
private boolean running = true; // Thread might never see false

// With volatile - always reads from main memory
private volatile boolean running = true; // Always sees latest value
```

Volatile visibility guarantee deta hai - but atomicity nahi. Counter ke liye AtomicInteger use

karo.

Exception Handling (Top 6)

#	Question	Category	Frequency
1	Checked vs Unchecked exceptions - difference?	Basics	Very High

2	throw vs throws - difference?	Keywords	High
3	try-with-resources kya hai?	Java 7+	High
4	Custom exception kaise banate hain?	Practical	High
5	finally block kab execute NAHI hota?	Tricky	High
6	Exception hierarchy explain karo	Basics	Medium

Key Answers

Checked vs Unchecked:

Type	Checked	Unchecked
------	---------	-----------

Type	Checked	Unchecked
Check hota hai	Compile time	Runtime
Handle karna	Mandatory (try-catch ya throws)	Optional
Extends	Exception	RuntimeException
Examples	IOException, SQLException	NullPointerException, ArrayIndexOutOfBoundsException
Kab use karein	Recoverable errors (file not found)	Programming bugs (null access)

finally kab execute NAHI hota:

1. System.exit() call ho
2. JVM crash ho
3. Thread kill ho (Thread.stop - deprecated)
4. Infinite loop in try/catch

Java 8+ Features (Top 8)

#	Question	Category	Frequency
1	Lambda expressions kya hain? Syntax?	Functional	Very High
2	Functional Interface kya hai? @FunctionalInterface?	Functional	Very High

3	Stream API - intermediate vs terminal operations?	Streams	Every interview
4	Optional class - kyun use karte hain?	Null safety	High
5	Method references kya hain? Types?	Functional	High
6	Predicate, Function, Consumer, Supplier - explain karo	Functional	High
7	Stream me map vs flatMap - difference?	Streams	Very High
8	Default methods in interfaces - kyun add hue?	Interfaces	Medium

Key Answers

Stream operations:

```
List<String> names = Arrays.asList("Aakash", "Rahul", "Amit", "Priya");

// Intermediate (lazy - execute nahi hote jab tak terminal na aaye)
names.stream()
    .filter(n -> n.length() > 4) // filter
    .map(String::toUpperCase) // transform
    .sorted() // sort

// Terminal (trigger execution)
    .collect(Collectors.toList()); // collect
    .forEach(System.out::println); // iterate
    .count(); // count
    .findFirst(); // first match
```

map vs flatMap:

```
// map - 1:1 transformation
List<String> words = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
// ["AAKASH", "RAHUL", "AMIT", "PRIYA"]

// flatMap - 1:many + flatten
List<List<Integer>> nested = Arrays.asList(
    Arrays.asList(1, 2), Arrays.asList(3, 4)
);
```



```
List<Integer> flat = nested.stream()
    .flatMap(Collection::stream)
    .collect(Collectors.toList());
// [1, 2, 3, 4]
```

JVM & Memory (Top 6)

#	Question	Category	Frequency
1	JVM architecture explain karo	Internals	High
2	Stack vs Heap memory - kya store hota hai kahan?	Memory	Very High
3	Garbage Collection kaise kaam karta hai?	GC	Very High
4	Types of GC (Serial, Parallel, G1, ZGC)	GC	Medium
5	Memory leak Java me kaise hota hai?	Problems	High
6	ClassLoader kya hai? Types?	Internals	Medium

Key Answers

Stack vs Heap:

Feature	Stack	Heap
Kya store hota hai	Local variables, method calls, primitives	Objects, instance variables
Access	Sirf us thread ko	All threads (shared)
Size	Small, fixed	Large, dynamic
Speed	Fast	Slower
Error	StackOverflowError	OutOfMemoryError
Management	Automatic (method return pe free)	Garbage Collector

Garbage Collection basics:

1. Mark - live objects mark karo (reachable from GC roots)
2. Sweep - unmarked objects delete karo
3. Compact - free space ko consolidate karo (fragmentation avoid)

GC Roots: static variables, active threads, local variables on stack, JNI

references **Memory leak causes:**

1. Static collections me objects add karte raho, remove nahi karo
2. Listeners/callbacks register karo, unregister na karo
3. Inner class holds reference to outer class (anonymous classes)
4. Unclosed resources (streams, connections)
5. Custom equals/hashCode galat - objects HashMap me phanse rehte hain

Design Patterns (Top 6)

#	Question	Category	Frequency
1	Singleton pattern - thread-safe kaise banayein?	Creational	Every interview
2	Factory pattern - kab use karein? Example?	Creational	Very High
3	Builder pattern - kab use karein?	Creational	High
4	Observer pattern explain karo	Behavioral	High
5	Strategy pattern explain karo	Behavioral	High

#	Question	Category	Frequency
6	Adapter pattern - real world example?	Structural	Medium

Key Answers

Singleton - Thread-safe (3 ways):

```
// 1. Eager initialization (simplest)
public class Singleton {
    private static final Singleton INSTANCE = new Singleton();
    private Singleton() {}
    public static Singleton getInstance() { return INSTANCE; } }
```

```
// 2. Double-checked locking
public class Singleton {
    private static volatile Singleton instance;
```

```

private Singleton() {}
public static Singleton getInstance() {
    if (instance == null) {
        synchronized (Singleton.class) {
            if (instance == null) {
                instance = new Singleton();
            }
        }
    }
    return instance;
}

// 3. Enum (BEST - recommended by Joshua Bloch)
public enum Singleton {
    INSTANCE;
    public void doSomething() { }
}

```

Factory pattern:

```

// Payment gateway - different implementations
public interface PaymentGateway {
    void pay(double amount);
}
public class UPIPayment implements PaymentGateway { ... }
public class CardPayment implements PaymentGateway { ... }
public class WalletPayment implements PaymentGateway { ... }

public class PaymentFactory {
    public static PaymentGateway create(String type) {
        return switch (type) {
            case "UPI" -> new UPIPayment();
            case "CARD" -> new CardPayment();
            case "WALLET" -> new WalletPayment();
            default -> throw new IllegalArgumentException("Unknown: " + type);
        }
    }
}

```

SOLID Principles (Top 5)

#	Question	Frequency
1	Single Responsibility Principle - explain with example	Very High
2	Open/Closed Principle - explain with example	High
3	Liskov Substitution Principle - explain simply	High
4	Interface Segregation Principle - explain	Medium
5	Dependency Inversion Principle - explain	High

Key Answers

Principle	One-line	Example
SRP	Ek class ka ek hi kaam hona chahiye	UserService sirf user logic, EmailService sirf email
OCP	Extension ke liye open, modification ke liye closed	New payment add karo bina existing code change kiye (use interface)
LSP	Child class parent ki jagah use ho sake bina break kiye	Square extends Rectangle → getArea() break karta hai (violation)
ISP	Fat interface mat banao, chhoti interfaces banao	Printer interface me scan() mat daalo agar sab printers scan nahi karte

Principle	One-line	Example
DIP	High-level module low-level pe depend na kare, dono abstraction pe depend karein	Service directly Repository pe depend na kare, interface ke through kare

Tricky Output Questions (Top 5)

```
// 1. String Pool
String a = "hello";
```

```
String b = "hello";
String c = new String("hello");
System.out.println(a == b); // true
System.out.println(a == c); // false
System.out.println(a.equals(c)); // true

// 2. Integer Cache
Integer x = 127, y = 127;
Integer p = 128, q = 128;
System.out.println(x == y); // true (cached)
System.out.println(p == q); // false (different objects)

// 3. Finally override
public static int test() {
    try { return 1; }
    finally { return 2; }
}
System.out.println(test()); // 2 (finally overrides return)

// 4. String concatenation
String s = "hello";
s.concat(" world");
System.out.println(s); // "hello" (Strings immutable, concat returns new)

// 5. Post-increment
int a = 5;
int b = a++;
System.out.println(a + " " + b); // 6 5 (post-increment returns old value)
```

How to Prepare (Strategy)

Week	Focus	Daily Target
Week 1	Core Java + OOPs (Q 1-22)	4 questions/day - code examples ke saath
Week 2	Collections + Multithreading (Q 23-42)	4 questions/day + implement karo
Week 3	Exceptions + Java 8 + JVM (Q 43-56)	4 questions/day + streams practice

Week 4	Design Patterns + SOLID + Tricky outputs + Mock interviews	3 questions + peer mocks
-----------	---	--------------------------

Golden Rules:

1. Sirf padhna nahi - code likho har answer ka
2. HashMap internal working - ye ek question 5 follow-ups generate karta hai, deep jaao
3. Multithreading - deadlock example khud likho, sirf theory mat padho
4. Output questions - roz 3 solve karo jab tak instant predict na kar pao
5. "I don't know" > galat answer - lekin curiosity dikhao

Beyond the Degree - Ye 50+ questions cover karo properly. Java interviews confident feel karoge guaranteed.